

MCI_ADV User Manual

Mass Casualty Incident Advanced Simulation Platform

Version 1.0 | April 2026

Table of Contents

- 1. [Overview](#)
- 2. [System Requirements](#)
- 3. [Installation](#)
- 4. [Project Structure](#)
- 5. [Quick Start Guide](#)
- 6. [Scenario Generation](#)
- 7. [Simulation Engine](#)
- 8. [Dashboard \(Streamlit\)](#)
- 9. [Batch Experiment Pipeline](#)
- 10. [Policy Combinations \(64 Scenarios\)](#)
- 11. [Evaluation Metrics](#)
- 12. [API Configuration](#)
- 13. [Troubleshooting](#)
- 14. [Appendix](#)

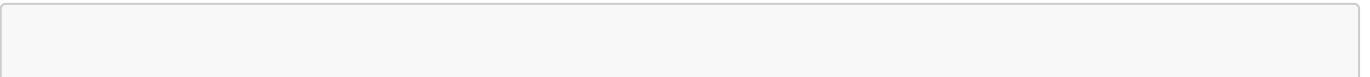
1. Overview

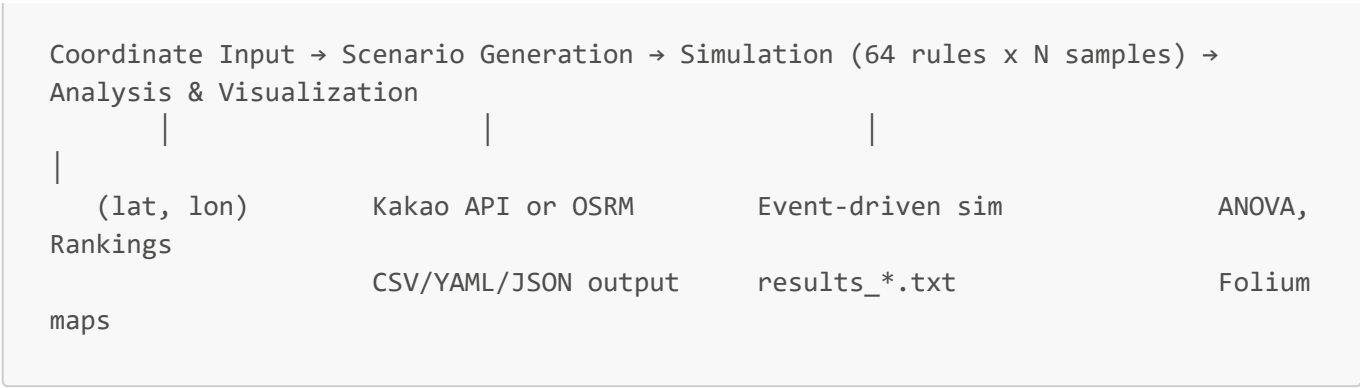
MCI_ADV is a simulation and analysis platform for optimizing patient transport in Mass Casualty Incidents (MCI). It evaluates 64 different dispatch policy combinations using ambulances (AMB) and unmanned aerial vehicles (UAV) to determine the optimal strategy for a given incident location.

Key Features

- **Scenario Generation:** Two road-data backends supported
 - **Kakao Mobility API** (`is_use_time=True`) — real-time/future traffic, requires a paid Korea-only key
 - **OSRM** (`is_use_time=False`) — open-source routing engine, no Kakao key required. Static road graph only (time computed as `distance/velocity`). Recommended for external reviewers and public users.
- **Simulation Engine:** Discrete event simulation with 64 rule combinations (Full Factorial Design)
- **Statistical Analysis:** ANOVA, Tukey HSD, Games-Howell post-hoc tests
- **Web Dashboard:** Interactive Streamlit-based UI with maps, analytics, and data editing
- **Batch Processing:** Large-scale experiments across hundreds of coordinates

System Pipeline





2. System Requirements

Hardware

- CPU: Multi-core recommended for batch experiments
- RAM: 8GB minimum, 16GB recommended for large-scale experiments
- Storage: ~500MB per 1000-coordinate experiment

Software

- Python 3.9+
- Windows 10/11, macOS, or Linux
- Internet connection (for road-data API calls — Kakao or OSRM)

API Key / Routing Backend (choose one)

Option A — Kakao Mobility API (is_use_time=True)

- **Kakao REST API Key** with the following services enabled:
 - Kakao Mobility (Directions API) — road distance + real-time/future duration
 - Kakao Local (Keyword/Address Search) — coordinate search / reverse-geocoding

Option B — OSRM backend (is_use_time=False)

- **No Kakao key required.** Recommended for external reviewers and public users.
- Uses [OSRM](#) HTTP API to fetch road distance + duration.
- Defaults to the official demo server (<https://router.project-osrm.org>), but its fair-use policy limits sustained traffic — self-host with docker for production. See the "OSRM Backend" section in the project root [README.md](#).
- The coordinate search / reverse-geocoding UI may not work in this mode (depends on Kakao Local). Direct CSV-based batch entry works fine.

3. Installation

3.1 Clone Repository

```
git clone <repository-url>
cd MCI_ADV
```

3.2 Install Dependencies

```
pip install -r requirements.txt
```

Key packages:

- `streamlit` (≥ 1.34) - Web dashboard
- `folium`, `streamlit-folium` - Map rendering
- `pandas`, `numpy` - Data processing
- `scipy`, `statsmodels` - Statistical analysis
- `pyyaml` - Configuration files
- `requests` - API calls
- `haversine` - Euclidean distance calculations
- `openpyxl` - Excel file I/O
- `geopandas`, `shapely` - Geographic boundary processing (batch experiments)

3.3 Required Data Files

Place the following files in the `scenarios/` directory:

File	Description	Required
<code>scenarios/안전센터와 소방서.csv</code>	Fire station/119 safety center master data	Yes
<code>scenarios/엑셀 결합 데이터.xlsx</code>	Hospital master data (beds, ORs, helipad, grades)	Yes
<code>scenarios/DISTANCE_MATRIX_FINAL.xlsx</code>	Pre-computed distance matrix	Optional

4. Project Structure

```
MCI_ADV/
├── src/
│   ├── sce_src/                                # Scenario Generation Module
│   │   ├── orchestrator.py                    # Master orchestrator (generate + simulate)
│   │   └── make_csv_yaml_dynamic.py          # Scenario data generator (API calls,
├── CSV/YAML)
│   └── BatchLab.py                            # Legacy batch processor
├── sim_src/                                    # Simulation Engine
│   ├── main.py                               # Entry point (RunManager)
│   ├── ScenarioManager.py                    # Entity initialization from config
│   ├── EntityManager.py                     # Entity state management
│   ├── EventManager.py                      # Event queue & simulation loop
│   ├── RuleManager.py                       # 64 policy rule definitions
│   └── MCIEnvironment_gymnasium.py          # Gymnasium environment wrapper
```

```

├── vis_src/                                # Dashboard & Visualization
│   ├── MCI_Streamlit.py                    # Main dashboard (3500+ lines)
│   └── pages/
│       ├── Generate.py                    # Scenario generation UI
│       ├── ResultsCompare.py              # Multi-coordinate result comparison
│       └── BatchExperiment.py             # Batch experiment dashboard
├── scenarios/                              # Generated scenario data
├── results/                               # Simulation results
├── experiment_logs/                       # Execution logs
├── experiment_1/                          # Batch experiment pipeline
└── requirements.txt

```

5. Quick Start Guide

5.1 Single Coordinate (Dashboard)

1. Launch dashboard:

```
streamlit run src/vis_src/MCI_Streamlit.py
```

2. Navigate to Generate page (sidebar)

3. Enter settings:

- Project path (auto-detected)
- Kakao REST API key (when **is_use_time** is checked) **or** OSRM URL (when unchecked)
- Departure date/time (Kakao mode only)
- Incident coordinates (search or manual input)

4. Configure parameters:

- Incident size (patients): default 30
- Ambulance count: default 30
- UAV count: default 3
- AMB speed: 40 km/h
- UAV speed: 80 km/h
- Handover time: AMB 10 min / UAV 15 min

5. Click "Generate & Run" → Scenario generation + simulation executes

6. View results in the main dashboard:

- **Scenarios tab**: Event logs, patient timelines
- **Maps tab**: Route visualization on Folium map
- **Analytics tab**: ANOVA, scenario rankings

5.2 Command Line (Single Coordinate)

```
# Step 1: Generate scenario - Kakao mode
python src/sce_src/make_csv_yaml_dynamic.py \
  --base_path . \
  --latitude 37.5665 --longitude 126.9780 \
  --is_use_time true \
  --kakao_api_key YOUR_API_KEY \
  --departure_time 202604031400 \
  --incident_size 30 --amb_count 30 --uav_count 3

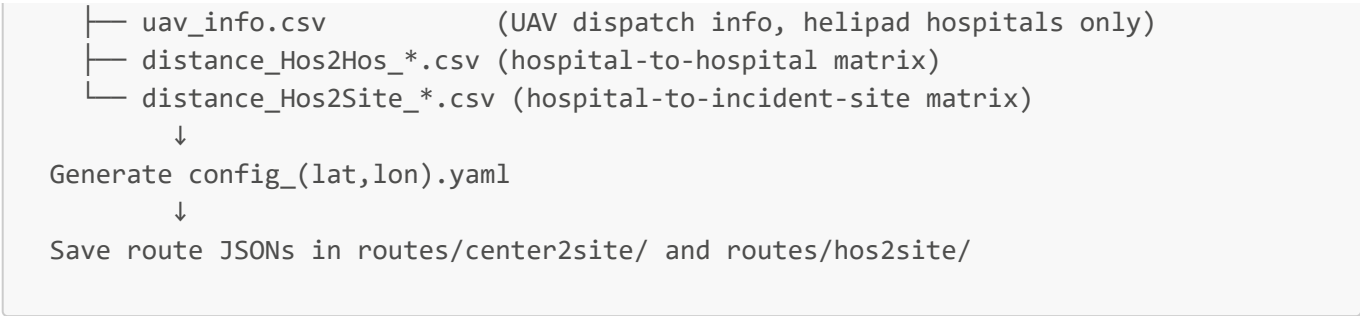
# Step 1 (alternative): Generate scenario - OSRM mode (no Kakao key)
python src/sce_src/make_csv_yaml_dynamic.py \
  --base_path . \
  --latitude 37.5665 --longitude 126.9780 \
  --is_use_time false \
  --osrm_url http://localhost:5000 \
  --incident_size 30 --amb_count 30 --uav_count 3

# Step 2: Run simulation
python src/sim_src/main.py --config_path scenarios/exp_.../config_(lat,lon).yaml
```

6. Scenario Generation

6.1 Process Flow

```
Input coordinates (lat, lon)
↓
Load hospital master data (엑셀 결합 데이터.xlsx)
↓
Filter hospitals within search radius
├─ Tier 3 (상급종합): grade code = 1
├─ Tier 2 (종합):      grade code = 11
└─ Tier 1 (병원):      grade code = 21, 28, 29, 31
↓
Kakao Mobility API or OSRM → road distance & duration for each hospital
↓
Load fire station data (안전센터와 소방서.csv)
↓
Select 30 nearest fire stations (Euclidean distance)
↓
Kakao Mobility API or OSRM → road distance & duration for each station
↓
Generate CSV files:
├─ patient_info.csv      (severity distribution: Green/Yellow/Red)
├─ hospital_info_road.csv (hospital list with road distances)
├─ hospital_info_euc.csv  (hospital list with straight-line distances)
├─ amb_info_road.csv      (ambulance dispatch info, road)
└─ amb_info_euc.csv       (ambulance dispatch info, euclidean)
```



6.2 Key Parameters

Parameter	Default	Description
incident_size	30	Number of patients
amb_count	30	Number of ambulances
uav_count	3	Number of UAVs
amb_velocity	40	Ambulance speed (km/h)
uav_velocity	80	UAV speed (km/h)
amb_handover_time	10.0	AMB patient handover time (min)
uav_handover_time	15.0	UAV patient handover time (min)
buffer_ratio	1.5	Hospital search radius multiplier
is_use_time	True	True: Kakao API duration. False: OSRM static distance with <code>distance/velocity</code> . In False mode the OSRM duration is also persisted to CSV, so re-running the same scenario folder with <code>is_use_time=True</code> will reuse the OSRM-derived duration.
osrm_url	(env <code>MCI_OSRM_URL</code> or <code>https://router.project-osrm.org</code>)	OSRM HTTP API base URL. Used only when <code>is_use_time=False</code> . The demo server has fair-use limits — self-hosting via docker is recommended.
duration_coeff	1.0	Duration weight coefficient
total_samples	30	Simulation repetitions per rule
random_seed	0	Random seed for reproducibility

6.3 Patient Severity Distribution

Patients are randomly assigned severity based on:

Severity	Proportion	Survival Model	Tier Restriction
Green	~60%	High survival, slow decay	Tier 1, 2, 3

Severity	Proportion	Survival Model	Tier Restriction
Yellow	~20%	Medium survival, moderate decay	Tier 2, 3
Red	~20%	Low survival, rapid decay	Tier 3 only

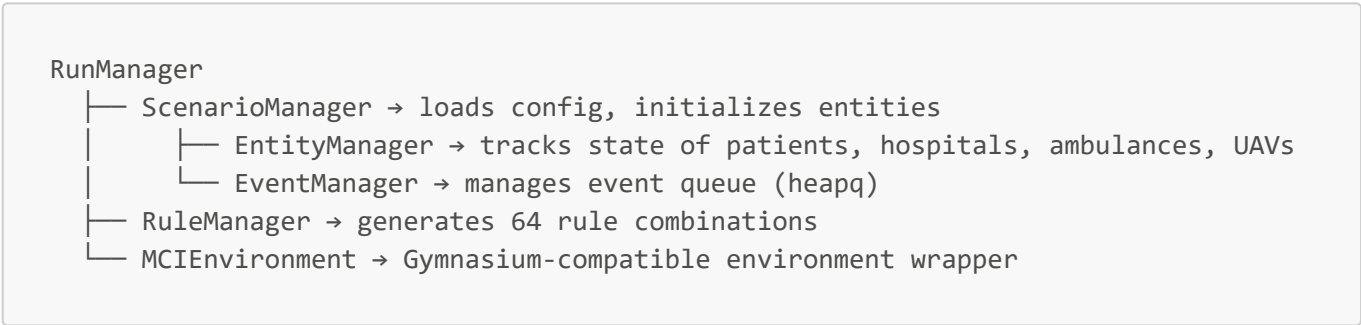
6.4 Hospital Tier System

Tier	Korean Name	Grade Codes	Capabilities
Tier 3	상급종합병원	1	All severity levels, helipad (some)
Tier 2	종합병원	11	Green + Yellow only
Tier 1	병원/요양병원	21, 28, 29, 31	Green only

7. Simulation Engine

7.1 Architecture

The simulation uses a **discrete event simulation (DES)** approach with a priority event queue.



7.2 Event Types

Event	Description
onset	Incident occurs, patients need rescue
p_rescue	Patient is rescued (severity assessed)
amb_arrival_site	Ambulance arrives at incident site
amb_arrival_hospital	Ambulance arrives at hospital
uav_arrival_site	UAV arrives at incident site
uav_arrival_hospital	UAV arrives at hospital
treat_start	Treatment begins at hospital
treat_end	Treatment complete

7.3 Simulation Loop

For each rule (64 total) x each sample (N repetitions):

1. `env.reset()` - Initialize scenario
2. Process events in chronological order via `EventManager.run_next()`
3. Rule selects action: which patient to transport, to which hospital, by which vehicle
4. Calculate survival probability based on elapsed time
5. Record metrics: Reward, Time, PDR, Reward_woG, PDR_woG

7.4 Diversion Rule

When a hospital reaches capacity, patients are diverted:

- AMB patients: transferred to nearest available hospital (any tier matching patient severity)
- UAV+Red patients: must go to helipad-equipped Tier 3 hospital only

8. Dashboard (Streamlit)

8.1 Launching

```
streamlit run src/vis_src/MCI_Streamlit.py
```

The dashboard opens at <http://localhost:8501>.

8.2 Main Page Tabs

Settings (Sidebar)

- Project path selection
- Experiment ID selection
- Coordinate selection
- Mini-map showing current coordinate location

Scenarios Tab

- **Experiment logs viewer:** Filter by coordinate
- **Patient timeline:** Rescue time, transport vehicle, hospital, arrival, treatment completion
- **Event table:** Full event log with Rule/Iteration filter

Maps Tab

- **Interactive Folium map** with route visualization
- **Ambulance routes:** Fire station -> Incident site (purple), Incident site -> Hospital (teal)
- **UAV routes:** Helipad hospital -> Incident site (dispatch), Incident site -> Hospital (transport)
- **Traffic congestion colors:** Clear (green), Moderate (blue), Slow (yellow), Congested (red)
- **Route info popups:** Distance (km), duration (min)
- Multi-select for displayed routes (limit to prevent rendering slowdown)

Analytics Tab

- **RAW results table:** Per-run values for each metric
- **STAT summary:** Mean, StdDev, 95% CI for all 64 scenarios
- **Scenario ranking:** Sort by Reward (desc), PDR (asc), or Time (asc)
- **ANOVA suite:** Full Factorial, One-way, RCBD analysis
- **Post-hoc tests:** Tukey HSD, Games-Howell
- **Residual diagnostics:** Shapiro-Wilk normality test, QQ plot, histogram

Data Tables Tab

- View and edit scenario CSV files directly
- Automatic backup before modification
- Re-run simulation with modified data

Rerun Tab

- Re-run simulation using existing YAML configuration
- Useful after editing scenario data

8.3 Additional Pages

Generate Page ([pages/Generate.py](#))

- Full scenario generation workflow
- Kakao API coordinate search (keyword + address)
- Batch coordinate input (multiple locations)
- Parameter preset management
- Generate + simulate in one click

Results Compare ([pages/ResultsCompare.py](#))

- Compare results across multiple coordinates
- Side-by-side metric comparison

Batch Experiment ([pages/BatchExperiment.py](#))

- 5-step workflow: Generate Coords -> View -> Run -> Progress -> Visualize
- Import functions from experiment_1 scripts
- Progress tracking with stop/resume capability

9. Batch Experiment Pipeline

9.1 Overview

The [experiment_1/](#) directory contains scripts for large-scale experiments across multiple coordinates.

9.2 Step-by-Step

Step 1: Generate Random Coordinates

```
python experiment_1/generate_coords.py \  
  --n 1000 --seed 42 \  
  --shp experiment_1/ctprvn.shp \  
  --out experiment_1/coords_korea.csv
```

- Generates N random coordinates within South Korea's land boundary
- Uses shapefile for precise boundary detection
- Outputs CSV with **coord_id**, **lat**, **lon** and preview HTML map

Step 2: Run Batch Processing

Kakao mode (real-time traffic)

```
python experiment_1/batch_runner.py \  
  --coords experiment_1/coords_korea.csv \  
  --kakao-api-key YOUR_API_KEY \  
  --experiment-id exp_korea_random_1000 \  
  --departure-time 202603311400 \  
  --daily-limit 4900 \  
  --total-samples 30
```

OSRM mode (open-source, no Kakao key)

```
python experiment_1/batch_runner.py \  
  --coords experiment_1/coords_korea.csv \  
  --is-use-time false \  
  --osrm-url http://localhost:5000 \  
  --experiment-id exp_korea_random_1000_osrm \  
  --total-samples 30
```

Key features:

- **Progress tracking:** **progress.json** saves state after each coordinate
- **Resume capability:** Automatically skips completed coordinates on restart
- **API quota management:** Tracks daily API calls, pauses when limit reached
- **Error handling:** Failed coordinates can be retried with **--max-retries**

Step 3: Visualize Results

```
python experiment_1/visualize_coords.py \  
  --coords experiment_1/coords_korea.csv \  
  --progress experiment_1/progress.json \  
  --clip-pct 2 --outlier-n 5
```

Outputs (saved in `scenarios/{experiment_id}/`):

- `coords_map.html`: Interactive result map (Reward/Time/PDR toggle, OpenStreetMap/CartoDB tile switch)
- `coords_map_hist.pdf/png`: Histogram distributions of key metrics
- `coords_map_rule_heatmap.pdf/png`: 64-rule performance heatmap (3 metrics × 4 panels)
- `coords_map_rule_effects.pdf/png`: Factor main effects chart (Best level marker, effect size box)

9.3 Progress JSON Structure

```
{
  "experiment_id": "exp_korea_random_1000",
  "total": 1000,
  "statuses": {
    "0": {
      "status": "done",
      "sim_ok": true,
      "coord": "(lat,lon)",
      "api_calls": 45,
      "error": null
    }
  },
  "api_log": {
    "2026-03-31": 2450,
    "2026-04-01": 2400
  }
}
```

10. Policy Combinations (64 Scenarios)

The simulation evaluates **64 rule combinations** using Full Factorial Design:

Factors

Factor	Levels	Description
Priority (Phase)	2	<code>START</code> (initial dispatch), <code>ReSTART</code> (re-evaluation)
Hospital Selection	2	<code>RedOnly</code> (Red patients first), <code>YellowNearest</code> (nearest available)
Red Action	4	<code>OnlyUAV</code> , <code>Both_UAVFirst</code> , <code>Both_AMBFirst</code> , <code>OnlyAMB</code>
Yellow Action	4	<code>OnlyUAV</code> , <code>Both_UAVFirst</code> , <code>Both_AMBFirst</code> , <code>OnlyAMB</code>

Total: 2 x 2 x 4 x 4 = **64 combinations**

Rule Naming Convention

```
{Phase}, {HospitalSelection}, Red {RedAction}, Yellow {YellowAction}
```

Example: **START**, **RedOnly**, **Red Both_UAVFirst**, **Yellow OnlyAMB**

11. Evaluation Metrics

Metric	Description	Better
Reward	Sum of survival probabilities for all patients	Higher
Time	Total simulation time (minutes)	Lower
PDR	Patient Death Rate (proportion of deaths)	Lower
Reward w.o.G	Reward excluding Green patients	Higher
PDR w.o.G	PDR excluding Green patients	Lower

Survival Probability Model

Each patient's survival probability decreases over time based on severity:

- **Green**: Slow decay (high baseline survival)
- **Yellow**: Moderate decay
- **Red**: Rapid decay (critical - time-sensitive)

12. API Configuration

12.1 Kakao REST API Key Setup (**is_use_time=True** mode)

1. Visit [Kakao Developers](#)
2. Create an application
3. Enable services:
 - **Kakao Mobility** (Directions API)
 - **Kakao Local** (Search API)
4. Copy the **REST API Key**

12.2 Kakao Endpoints Used

Endpoint	Purpose	Calls per Coordinate
Mobility Directions	Road distance/duration for hospitals	~20-50
Mobility Directions	Road distance/duration for fire stations	~30
Local Keyword Search	Coordinate search (dashboard only)	1 per search
Local Address Search	Address lookup (dashboard only)	1 per search

12.3 Kakao Quota Management

- Kakao free tier: **5,000 calls/day**

- Each coordinate requires approximately **50-80 API calls**
- `batch_runner.py` tracks daily usage via `api_log` in `progress.json`
- Configure `--daily-limit` (default: 4900) to leave safety margin

12.4 OSRM Backend (`is_use_time=False` mode)

When you don't have a Kakao key, or when publishing the project, use OSRM (an open-source routing engine).

Default behavior: If the env var `MCI_OSRM_URL` is set it is used; otherwise the official demo server <https://router.project-osrm.org> is used. CLI/UI `--osrm_url` overrides explicitly.

Self-hosting (recommended for production):

```
# Download Korean OSM extract and pre-process
wget https://download.geofabrik.de/asia/south-korea-latest.osm.pbf
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-extract -p /opt/car.lua
/data/south-korea-latest.osm.pbf
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-partition /data/south-
korea-latest.osrm
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-customize /data/south-
korea-latest.osrm

# Start the routing server
docker run -t -i -p 5000:5000 -v "$(pwd):/data" osrm/osrm-backend \
  osrm-routed --algorithm mld /data/south-korea-latest.osrm

# Use it
export MCI_OSRM_URL=http://localhost:5000
```

OSRM endpoints used:

Endpoint	Purpose
<code>/route/v1/driving/{lon1},{lat1};{lon2},{lat2}</code>	Road distance + duration + GeoJSON polyline for hospitals/fire stations

A self-hosted OSRM instance has no external rate limit (only server resource limits). The `--daily-limit` flag remains for compatibility but is meaningless in OSRM mode.

Limitations:

- No real-time traffic data (static road graph)
- No road congestion data → dashboard map renders single-color polylines
- Coordinate search / reverse-geocoding UI depends on Kakao Local and is unavailable in OSRM mode. Use direct CSV batch input instead.

13. Troubleshooting

Common Issues

Problem	Cause	Solution
<code>RuntimeError</code> in scenario generation	Incident site has no nearby road (rc=102): mountain, sea, uninhabited island	Not a bug; coordinate is genuinely inaccessible by road. All 223/1000 failures in the batch experiment are this type. Fire station route issues: 0 cases.
<code>Exception: Impossible to divert</code>	Code logic bug: UAV+Red patient can only go to helipad+Tier3 hospitals. When the only qualifying hospital (e.g., Wonkwang Univ. Hospital, capacity=16) is full, diversion fails even though non-helipad Tier3 hospitals have capacity.	Known edge case (5/1000 coords affected). Not a bed shortage — total Tier3 capacity (35) exceeds patient count (30). Fix requires fallback logic in <code>diversion_rule()</code> .
API 401 Unauthorized	Invalid Kakao API key	Check key in Kakao Developer console. If no key is available, switch to <code>is_use_time=false</code> (OSRM backend).
API 429 Rate Limit	Kakao daily quota exceeded	Wait 24h, increase quota, or switch to the OSRM backend.
<code>RuntimeError: Kakao API key required (is_use_time=True mode)</code>	Generating with <code>is_use_time=True</code> but no <code>--kakao_api_key</code> provided	Provide a key, or use <code>is_use_time=false</code> (OSRM).
<code>OSRM 경로 없음 (code=NoRoute)</code>	Coordinate not connected in the OSRM road graph (sea/island)	Exclude the coordinate. Equivalent to Kakao rc=102.
<code>UnicodeEncodeError: cp949</code>	Windows console encoding	Set <code>PYTHONIOENCODING=utf-8</code>
YAML key order crash	<code>sort_keys=True</code> in <code>yaml.dump</code>	Use <code>sort_keys=False</code> (already fixed)
Simulation hangs	stdout pipe buffer full	Fixed via Popen + daemon thread

Performance Tips

- Batch experiments: Run overnight to maximize API quota usage
- Dashboard: Use Analytics "Load Analysis Data" button (lazy loading)
- Maps: Limit displayed routes to 10-15 for smooth rendering

Appendix

A. Config YAML Structure

```
entity_info:
  departure_time: "202604031400"
  patient:
    info_path: ./scenarios/exp_.../patient_info.csv
    total: 30
  hospital:
    info_road_path: ./scenarios/exp_.../hospital_info_road.csv
    info_euc_path: ./scenarios/exp_.../hospital_info_euc.csv
    distance_h2h_road_path: ./scenarios/exp_.../distance_Hos2Hos_road.csv
    distance_h2s_road_path: ./scenarios/exp_.../distance_Hos2Site_road.csv
  ambulance:
    info_road_path: ./scenarios/exp_.../amb_info_road.csv
    info_euc_path: ./scenarios/exp_.../amb_info_euc.csv
    velocity: 40
    handover_time: 10.0
  uav:
    info_path: ./scenarios/exp_.../uav_info.csv
    velocity: 80
    handover_time: 15.0

output_path: ./results/exp_.../
totalSamples: 30
randomSeed: 0
```

Important: The `entity_info` key order must be: `departure_time` -> `patient` -> `hospital` -> `ambulance` -> `uav`. Changing this order causes a crash because `ScenarioManager` processes entities in dict insertion order.

B. Result File Formats

RAW (`results_(lat,lon).txt`):

```
START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.5 13.2 11.8 ...
START, RedOnly, Red OnlyUAV, Yellow Both_UAVFirst 14.1 12.9 ...
...
```

Each line: rule label followed by N sample values. 5 metric blocks (Reward, Time, PDR, Reward_woG, PDR_woG), each containing 64 lines.

STAT (`results_(lat,lon)_stat.txt`):

```
START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.50 0.72 0.26
```

Each line: rule label followed by mean, standard deviation, 95% CI half-width.

C. Fire Station Data Format

안전센터와 소방서.csv (CP949 encoding):

```
기관명,주소,전화번호,x좌표,y좌표,관할구역,대수  
강남소방서,서울특별시 강남구...,02-3400-1119,127.0495,37.4979,...,5
```

D. Hospital Excel Format

엑셀 결합 데이터.xlsx:

- Key columns: 요양기관명, 종별코드, 수술실수, 병상수, 헬기장 여부, x좌표, y좌표, 주소, 전화번호

MCI_ADV - Mass Casualty Incident Advanced Simulation Platform

For questions or issues, please visit the [GitHub repository](#).